

Assessment Tool 1 – Answer (CO5)

Topic 3: AI-Based Fraud Detection Framework

1. Problem Understanding & Requirements

Financial institutions process millions of transactions per second across accounts, cards, and channels. The goal is a **scalable fraud detection system** that: (a) ingests high-volume transaction streams, (b) flags anomalous/suspicious transactions in near real time, (c) minimises false positives (to avoid blocking genuine customers) and false negatives (missed fraud), and (d) continuously improves as fraud patterns evolve. Constraints: low latency (sub-second scoring), high throughput (millions of records/day), auditability, and resilience to node failure.

2. System Architecture

[Transaction Sources] --> [Ingestion Layer] --> [PySpark Cluster: RDD/Streaming Processing]

(POS, ATM, Online, (Kafka / Flume) |-- Feature Engineering

Mobile banking) |-- Anomaly Detection Agents

 |-- Rule-Engine Agents

v

[Alerting & Case-Management Agent]

|

[Feedback Loop -> Model Retraining]

|

[Dashboard / Analyst Review]

- **Ingestion Layer:** streaming source (e.g., Kafka topics per channel) feeding Spark Structured Streaming / Spark Streaming DStreams, which internally use RDDs as micro-batches.
- **Processing Layer:** a PySpark cluster (driver + worker nodes) partitions transaction data across nodes for parallel processing.
- **Agent Layer:** intelligent agents (see Section 4) run atop the processed RDDs to score, correlate, and alert.
- **Storage:** a distributed store (HDFS/S3 + a fast key-value store like Redis/Cassandra for recent transaction history per account) supports both batch feature computation and low-latency lookups.

3. Use of PySpark & RDD Operations

- **Ingestion → RDD:** each micro-batch of transactions is loaded as an RDD of records (txn_id, acct_id, amount, location, timestamp, merchant).

- **map**: transform raw records into feature vectors (amount z-score, time-since-last-txn, geo-distance from last known location).
- **filter**: discard incomplete/malformed records before scoring.
- **reduceByKey**: aggregate per-account statistics (e.g., rolling transaction count, total spend in last hour) keyed by acct_id.
- **groupByKey / combineByKey**: build per-account transaction history windows for sequence-based anomaly checks.
- **join**: enrich transaction RDDs with reference RDDs (customer profile, merchant risk score, blacklist).
- **mapPartitions**: apply a trained anomaly-scoring model (e.g., Isolation Forest or Autoencoder loaded once per partition) efficiently across partitions.
- **persist()/cache()**: cache frequently reused RDDs (e.g., customer profile RDD) across streaming batches to avoid recomputation.
- **foreachPartition**: push high-score alerts to the alerting system without collecting all data to the driver (avoids bottlenecks).

4. Intelligent Agent Integration

- **Anomaly-Detection Agent**: a reactive/learning agent that scores each transaction against a statistical/ML anomaly model (Isolation Forest, One-Class SVM, or Autoencoder reconstruction error) computed per Spark partition; flags transactions above a dynamic threshold.
- **Rule-Engine Agent**: a rule-based reflex agent enforcing hard business rules (e.g., transaction > \$10,000 in a new country within 1 hour of prior transaction) — runs in parallel to the ML agent to catch known fraud typologies instantly.
- **Correlation/Case-Management Agent**: a goal-based agent that aggregates alerts from multiple accounts/agents to detect coordinated fraud rings (e.g., many small transactions from linked accounts), and decides whether to auto-block, hold for review, or escalate.
- **Learning/Feedback Agent**: continuously ingests analyst-confirmed labels (true fraud / false positive) and triggers periodic retraining of the anomaly model, so detection accuracy improves over time — this closes the loop between agents 1–3 and the offline PySpark training pipeline.
- Agents collaborate via a shared alert bus/message queue: the Rule-Engine and Anomaly agents publish candidate alerts; the Correlation Agent subscribes, cross-references, and produces the final decision; the Learning Agent subscribes to outcomes to retrain.

5. Scalability & Distributed Computing Design

- **Horizontal scaling**: adding worker nodes increases partition-level parallelism; transaction volume growth is handled by increasing partitions (repartition()) and cluster size.

- **Fault tolerance:** RDD lineage allows Spark to recompute lost partitions automatically after node failure, without data loss; checkpointing is used for long streaming lineages to bound recovery time.
- **Load balancing:** partitioning transactions by acct_id hash spreads load evenly and keeps a customer's history co-located on one partition for stateful checks.
- **Elastic scaling:** cluster managers (YARN/Kubernetes) can scale executor count based on incoming stream volume (e.g., peak shopping periods).

6. Data Flow & Processing Pipeline

1. Transaction event → Kafka topic → Spark Streaming micro-batch → RDD.
2. Feature engineering (map) → enrichment (join with profile/merchant RDDs).
3. Parallel scoring: Rule-Engine Agent (filter) + Anomaly Agent (mapPartitions with ML model).
4. Alerts merged and sent to Correlation Agent (reduceByKey on account/ring identifiers).
5. Final decision (allow / hold / block) pushed to case-management dashboard and transaction gateway.
6. Analyst feedback stored and periodically fed into offline PySpark MLlib retraining job (batch RDD/DataFrame pipeline).

7. Innovation & Creativity

- Combines a **fast rule-based reflex layer** with a **slower learning-based anomaly layer**, giving both immediate coverage of known fraud patterns and adaptive coverage of novel ones.
- **Graph-based ring detection:** representing accounts/transactions as a graph RDD (GraphFrames) to catch collusion patterns invisible to single-transaction scoring.
- **Human-in-the-loop feedback agent** ensures the system's false-positive rate keeps improving rather than degrading (concept-drift resilience).

8. Summary

This design shows how PySpark's distributed RDD operations (map, filter, reduceByKey, join, mapPartitions) enable real-time, large-scale transaction processing, while a layered set of intelligent agents (reflex rule agent, learning anomaly agent, goal-based correlation agent, feedback agent) collaborate to detect, escalate, and continuously improve fraud detection at scale.